1 Foundations: how models are trained

Months 1–2

WHAT YOU WILL DO

How learning works — loss, gradients, backpropagation — then language modeling and tokenization, and the transformer as a machine you can count. Every concept is implemented small before it is used big: each engineer writes the training loop, a BPE tokenizer, attention from scratch, and prices real models from their configs.

WHAT YOU WILL HAVE BUILT

A shared foundation across the team, with working implementations of the core mechanisms written by each engineer.

MATERIALS

BOOK — written

Ch. 1 Programs you don't write
Ch. 2 Language modeling
Ch. 3 The transformer, mechanically

INTERACTIVE

Gradient-descent playground · BPE stepper · Parameter counter · RoPE rotations

LABS

One Colab per chapter — run the book's numbers yourself

CH. 1

Programs you don't write

Loss, gradients, and the machine that turns data into functions

THE CHAPTER IN FIVE IDEAS

- You write the loss; the loss writes the program — the one mental shift the whole field rests on. Llama 2 7B is a function with 6.7B constants nobody wrote.
- The entire recipe in four lines — forward, loss, backward, update — run for real on a two-parameter rent model: loss 7,205,000 → 3,425 in ten steps, then the noise floor.
- Backpropagation is the chain rule over your call graph: the full gradient costs ~2 extra forward passes for any number of parameters — and its stored activations are what fill a training GPU.
- The optimizer minimizes training loss, which is a proxy: held-out data is the only honest measure, and at LLM scale the failure mode is benchmark contamination.
- Everything compiles to matrix multiplies — the hardware ladder from Python loops to an H100 spans ~7 orders of magnitude, and that is why GPUs exist.

6,738,415,616

constants in Llama 2 7B — not one chosen by a person. This chapter explains the machine that chose them.

GO DEEPER

Read: chapters/ch01_programs.html
Lab (Colab): labs/ch01_programs.ipynb
Play: gradient-descent playground — cause the divergence yourself

CH. 2

Language modeling

Tokens, cross-entropy, and the numbers you will stare at for months

THE CHAPTER IN FIVE IDEAS

- A tokenizer chops bytes into units by pure frequency (BPE): frequent strings become one token, rare ones stay in pieces — and an English-centric vocabulary taxes Spanish 30–50% more tokens per sentence.
- Embeddings turn ids into learned vectors whose geometry encodes similarity of usage — nobody defines similarity; prediction pressure does.
- The loss is one line: pay −log p of the true next token. Every position of every sequence trains at once; no labels are ever bought.
- Perplexity = e^loss is the effective branching factor: 50,257 at step zero, a cliff to ~50 on cheap statistics, then the expensive grind toward the entropy floor of language.
- Decoding (temperature, top-p) is not part of the model — the same weights are a careful assistant at T=0.5 and a brainstormer at T=1.2.

ln 50,257 = 10.8

the first loss every GPT-2-vocabulary run prints, before learning anything — uniform guessing over the vocabulary.

GO DEEPER

Read: chapters/ch02_language_modeling.html
Lab (Colab): labs/ch02_language_modeling.ipynb
Play: BPE, one merge at a time

CH. 3

The transformer, mechanically

What each block does, what it costs, and how to count it by hand

THE CHAPTER IN FIVE IDEAS

- The residual stream is a bus: every component reads it, computes an update, and adds it back — attention moves information between positions, the MLP thinks per position.
- Attention is a soft dictionary lookup: queries dot keys, softmax, weighted sum of values — with a causal mask and a √d_k that, if forgotten, silently ruins training.
- The MLP holds two-thirds of the parameters and works as a key-value memory: where the model's facts physically live.
- The modern block = the 2017 decoder minus cross-attention, plus four swaps: pre-norm, RMSNorm, RoPE, SwiGLU.
- Five config integers give the four facts: parameters $N(4d^2+3d{\cdot}d\_ff)+2Vd$ — 6.74B for Llama 2 7B, exact to the RMSNorm vectors — disk, FLOPs/token (6P), and KV cache (512 KB/token).

5 → 4

five integers in a config.json determine parameters, disk size, FLOPs per token, and KV-cache memory — computable in your head.

GO DEEPER

Read: chapters/ch03_transformer.html
Lab (Colab): labs/ch03_transformer.ipynb
Play: the parameter counter · RoPE rotations